

PSY40760 - Paths and environment instructions

Basic R/Rstudio understanding

If you are struggling with understanding how R and RStudio work, read Chapter 3 by D. Navarro: <https://learningstatisticswithr.com/03-introR.html>

Setting up RStudio and Quarto for PSY40760

This is an initial setup that you need to do which involves setting up the paths and environment and installing the relevant packages inside your environment (i.e. “Initial setup”).

Then, on every subsequent worksheet, you won’t have to repeat this setup. You can just load the already set-up environment.

Folders

The structure of your folder should follow what’s on GitHub (<https://github.com/ozika/psy40760-practicals-2026>)

During **initial setup**, you can download the entire folder from GitHub. However, on subsequent sessions, you should only download the relevant files for that session.

Why? If you download everything anew it would overwrite your work from past worksheets as well as the R (renv) environment that you have set up - i.e. you would have to do the setup again.

Each session has its own folder. The easiest way is to create the session folder (e.g. **seminar-01-intro-to-stats-and-R** - exact naming is *important*) and then download the individual files from GitHub. Note that the location of the files on your computer relative to the main folder should be identical, i.e. follow the folder and file structure from GitHub exactly.

An alternative, which is more complex to set up but *much* easier to use afterwards, is to use git. You need to install git first. Then set up SSH keys (<https://docs.github.com/en/authentication/connecting-to-github-with-ssh>). After that the entire GitHub folder can be cloned to your computer with `git clone git@github.com:ozika/psy40760-practicals-2026.git` (the command can be run from the command line or from the Terminal tab in RStudio). On subsequent sessions, when new materials become available, run `git pull` inside the folder to download them.

`git pull` stops with an error if a file you have changed yourself has also been updated on GitHub. So do your work in a copy of each worksheet (e.g. the file you submit, named with your student number) rather than in the original. If `git pull` still complains about `renv.lock` (because you added your own packages), run `git checkout -- renv.lock`, then `git pull`, then `renv::restore()`, and re-install your extra packages.

Telling R where you are working

R has no idea where your files are unless you tell it to. The quickest way is to either use

```
setwd("path/to/folder/psy40760-practicals-2026-main")
```

However, during our practical it turned out that the newest release of RStudio for Windows has a bug and will give an error if your path includes a Unicode character like `\U`.

The alternative:

Click **Session** on top. Go to **Set Working Directory and Choose directory....** Select the project folder `psy40760-practicals-2026-main`.

Initial setup of the environment

Our folder has a `renv.lock` file located in it. This is a recipe file which tells `renv` what packages should be installed.

As a prerequisite, the `renv` package should be installed and loaded into R.

To do this run:

```
# install renv (you might already have it)
install.packages("renv")

# load renv
library(renv)
```

You can read more about what `renv` is and how to use it here: <https://rstudio.github.io/renv/articles/renv.html>

Next, make sure your working directory is the course folder (see above), then load the project's environment and install the packages listed in `renv.lock`:

```
# switch this R session to the course's package library
renv::load()

# install the packages recorded in renv.lock
renv::restore()
```

This will ask you a few times to confirm (type y each time). You might also get a question about whether you want to re-install the environment, in which case say yes - this is typically option 4.

Setting up the paths

Now we set up another core package called **here**, which will help us manage our paths. If it is not yet in the environment, **renv** installs it:

```
# install here (does nothing if it is already installed)
renv::install("here")

# load here
library(here)
```

Next, we need to check that **here** knows where our project lives. To do this, we are using a hidden file called `.root-practicals` which is already located in your folder. See FAQs on how to see hidden files on your system.

To tell **here** where on our computer the project is we use:

```
here::i_am(".root-practicals")
```

This looks for the hidden file `.root-practicals` in your current working directory and in the folders above it, and notes the folder that contains it as our project folder (also known as 'root').

You can check that this has worked by typing:

```
here::here()
```

This should return the full correct path on your computer. If it doesn't, see the FAQs below.

You are all set up!

Working with renv

You have now set up and loaded your environment.

You can now install packages as you need, for example:

```
# install additional packages
renv::install("dplyr")
```

Important - this environment is now yours, and you need to also save it. After you install new packages, you need to run

```
renv::snapshot()
```

This will update your `renv.lock` file.

Using here and renv from now on

While the initial setup above is only done once, we will need to load the environment at the top of each script. Therefore each worksheet and script that you return from now on should

1. Use **here** to determine correct folder
2. Use **renv** to load the environment
3. Use **library()** to load the correct packages

Install new packages in the console (see “Working with renv” above), not in the worksheet: an install line inside a worksheet would run again every time it is rendered.

This means including the following (or similar) at the top of each script:

```
library(here)
here::i_am(".root-practicals")
here::here() #print path to check

library(renv)
renv::load(here::here())

library(dplyr) #and other packages that you might need
```

FAQs

When I run `here::i_am(".root-practicals")`, `here::here()` outputs the wrong folder (e.g. Downloads)

This is most likely caused by the fact that you have downloaded the scripts several times. There is a hidden file `.root-practicals` which is part of our shared folder (though it may not be visible through your normal file viewer). If you download the folder several times, `.root-practicals` will be present at several locations. You need to delete the other copies of it that you have downloaded.

How to see and remove hidden files on Mac/Windows

Windows: Open File Explorer (Windows key + E) then **File Explorer -> View -> Hidden items**

Mac: Open Finder. You can view hidden files by pressing Command (Cmd), Shift, and period (.) key.

Once you can see the `.root-practicals` hidden file, you can remove it from the wrong folder.

Why use `here::here()` rather than `here()`?

First what does `here::here()` actually mean? The first `here` is the name of the package, the second is the name of a function within the package. Here, not so smartly, they are the same. In most cases they will not be (e.g. `dplyr::select()`). So, why do this?

The specific functions (e.g. `select()`) often have the same names in different packages, if we just type `select()` R will use the function loaded first, which can be an entirely different function from the one we intended. Therefore, specifying the package name first is a great habit.